

IDMA 2024 Problem Set 2

Lauritz Tørholm Jørgensen, fqm630@alumni.ku.dk

June 2026

1

1a

We are given the array

$$A = [5, 6, 4, 7, 3, 8, 2, 9, 1, 10].$$

We run insertion sort from left to right. At the beginning of each step, the part of the array to the left of the current element is already sorted.

The initial array is

$$[5, 6, 4, 7, 3, 8, 2, 9, 1, 10].$$

For $j = 2$, the key is 6. Since $5 \leq 6$, no elements are moved. The array is still

$$[5, 6, 4, 7, 3, 8, 2, 9, 1, 10].$$

For $j = 3$, the key is 4. The elements 6 and 5 are both larger than 4, so they are moved one position to the right. Then 4 is inserted at the beginning:

$$[4, 5, 6, 7, 3, 8, 2, 9, 1, 10].$$

For $j = 4$, the key is 7. Since $6 \leq 7$, no elements are moved:

$$[4, 5, 6, 7, 3, 8, 2, 9, 1, 10].$$

For $j = 5$, the key is 3. The elements 7, 6, 5, 4 are moved one position to the right. Then 3 is inserted at the beginning:

$$[3, 4, 5, 6, 7, 8, 2, 9, 1, 10].$$

For $j = 6$, the key is 8. Since $7 \leq 8$, no elements are moved:

$$[3, 4, 5, 6, 7, 8, 2, 9, 1, 10].$$

For $j = 7$, the key is 2. The elements 8, 7, 6, 5, 4, 3 are moved one position to the right. Then 2 is inserted at the beginning:

$[2, 3, 4, 5, 6, 7, 8, 9, 1, 10]$.

For $j = 8$, the key is 9. Since $8 \leq 9$, no elements are moved:

$[2, 3, 4, 5, 6, 7, 8, 9, 1, 10]$.

For $j = 9$, the key is 1. The elements 9, 8, 7, 6, 5, 4, 3, 2 are moved one position to the right. Then 1 is inserted at the beginning:

$[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$.

For $j = 10$, the key is 10. Since $9 \leq 10$, no elements are moved:

$[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$.

So insertion sort returns

$[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$.

1b

We again start with

$A = [5, 6, 4, 7, 3, 8, 2, 9, 1, 10]$.

Merge sort first divides the array into two halves:

$[5, 6, 4, 7, 3]$ and $[8, 2, 9, 1, 10]$.

The left half is divided as

$[5, 6, 4]$ and $[7, 3]$.

The first part $[5, 6, 4]$ is divided as

$[5, 6]$ and $[4]$.

Then $[5, 6]$ is divided into

$[5]$ and $[6]$.

These are single-element arrays, so they are already sorted. Merging them gives

$[5, 6]$.

Now we merge $[5, 6]$ with $[4]$. The element 4 is smallest, so the result is

$[4, 5, 6]$.

The other part of the left half is

$[7, 3]$.

It is divided into

$[7]$ and $[3]$.

Merging these gives

$[3, 7]$.

Now the two sorted parts of the left half are

$[4, 5, 6]$ and $[3, 7]$.

Merging them gives

$[3, 4, 5, 6, 7]$.

Now we sort the right half

$[8, 2, 9, 1, 10]$.

It is divided into

$[8, 2, 9]$ and $[1, 10]$.

The first part $[8, 2, 9]$ is divided into

$[8, 2]$ and $[9]$.

Then $[8, 2]$ is divided into

$[8]$ and $[2]$.

Merging these gives

$[2, 8]$.

Now we merge $[2, 8]$ with $[9]$, giving

$[2, 8, 9]$.

The other part of the right half is

$[1, 10]$.

It is divided into

$[1]$ and $[10]$.

Merging these gives

$[1, 10]$.

Now the two sorted parts of the right half are

$[2, 8, 9]$ and $[1, 10]$.

Merging them gives

$[1, 2, 8, 9, 10]$.

Finally, the two sorted halves of the original array are

$[3, 4, 5, 6, 7]$ and $[1, 2, 8, 9, 10]$.

Merging them gives

$[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$.

So merge sort also returns

$[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$.

1c

Now suppose that the array B is already sorted in increasing order.

For insertion sort, this is the best case. When the algorithm considers a new key, the key is already at least as large as the element just before it. Therefore no elements have to be moved to the right.

The algorithm still has to scan through the array once, so the running time is

$\Theta(n)$.

For merge sort, the algorithm still divides the array into smaller arrays and then merges them again. The input order does not change the number of recursive calls or the amount of merging work in the standard version of merge sort. Therefore the running time is

$\Theta(n \log n)$.

Thus insertion sort is asymptotically faster in this case, since

$\Theta(n)$

is asymptotically smaller than

$\Theta(n \log n)$.

1d

Now suppose that the array C is sorted in decreasing order, but we want to sort it in increasing order.

For insertion sort, this is the worst case. Every new key is smaller than all elements before it, so all previously sorted elements have to be moved one position to the right.

The number of moved elements is

$$1 + 2 + \cdots + (n - 1) = \frac{n(n - 1)}{2}.$$

Therefore the running time of insertion sort is

$$\Theta(n^2).$$

For merge sort, the running time is still

$$\Theta(n \log n).$$

The reason is the same as before: merge sort still splits the array recursively and does linear merging work at each level.

Thus merge sort is asymptotically faster in this case, since

$$\Theta(n \log n)$$

is asymptotically smaller than

$$\Theta(n^2).$$

2

2a

The claim has to be true by the pigeonhole principle.

There were more than 100 participants at the conference. Suppose, for contradiction, that both of the following statements are false:

- the participants came from more than 10 different countries,
- more than 10 participants came from some particular country.

Then the participants came from at most 10 different countries.

Also, since no country had more than 10 participants, each of these countries contributed at most 10 participants.

Therefore the total number of participants would be at most

$$10 \cdot 10 = 100.$$

But this contradicts the fact that the conference had more than 100 participants.

Hence at least one of the two statements must be true. So either the participants came from more than 10 different countries, or some country had more than 10 participants.

2b

The only possible final outcome is

$$191.$$

To see this, consider the sum of all numbers currently written on the sheet. At the beginning, the numbers are

$$1, 2, \dots, 20.$$

Their sum is

$$1 + 2 + \dots + 20 = \frac{20 \cdot 21}{2} = 210.$$

In one move, two numbers a and b are erased and replaced by

$$a + b - 1.$$

Before the move, these two numbers contributed

$$a + b$$

to the total sum. After the move, they contribute

$$a + b - 1.$$

So every move decreases the total sum by exactly 1.

The game starts with 20 numbers and ends with 1 number. Since each move reduces the number of entries by 1, the children make exactly

$$20 - 1 = 19$$

moves.

Therefore the final sum is

$$210 - 19 = 191.$$

When there is only one number left, that number is equal to the sum of all numbers on the sheet. Hence the final number must be

$$191.$$

This does not depend on which numbers are chosen during the game. The range of possible outcomes is therefore just

$$\{191\}.$$

3

3a

We prove the statement by induction on s .

For $s = 0$, we have

$$1 + 0 \cdot k = 1$$

and

$$(1 + k)^0 = 1.$$

So the statement is true for $s = 0$.

Now suppose that the statement is true for some $s \in \mathbb{N}$. That is, suppose that

$$1 + sk \leq (1 + k)^s.$$

We want to prove that

$$1 + (s + 1)k \leq (1 + k)^{s+1}.$$

Using the induction hypothesis,

$$(1 + k)^{s+1} = (1 + k)^s(1 + k) \geq (1 + sk)(1 + k).$$

Expanding the right-hand side gives

$$(1 + sk)(1 + k) = 1 + k + sk + sk^2 = 1 + (s + 1)k + sk^2.$$

Since $s \geq 0$ and $k > 0$, we have

$$sk^2 \geq 0.$$

Therefore

$$1 + (s + 1)k + sk^2 \geq 1 + (s + 1)k.$$

So

$$(1 + k)^{s+1} \geq 1 + (s + 1)k.$$

This proves the induction step. Hence, for all $s \in \mathbb{N}$ and all $k > 0$,

$$1 + sk \leq (1 + k)^s.$$

3b

The sequence is defined by

$$T_1 = 0, \quad T_2 = 1, \quad T_i = T_{i-1} + T_{i-2} \quad \text{for } i \geq 3.$$

We first note that

$$T_i > 0$$

for all $i \geq 2$. This is clear for $i = 2$, since $T_2 = 1$. After that, each term is the sum of two earlier nonnegative terms, and at least one of them is positive. So the fractions below are well-defined.

We prove by induction on i that

$$1 \leq \frac{T_{i+1}}{T_i} \leq 2$$

for all $i \geq 2$.

For $i = 2$, we have

$$T_3 = T_2 + T_1 = 1 + 0 = 1.$$

Thus

$$\frac{T_3}{T_2} = \frac{1}{1} = 1.$$

So

$$1 \leq \frac{T_3}{T_2} \leq 2.$$

Now suppose that the statement is true for some $i \geq 2$. That is,

$$1 \leq \frac{T_{i+1}}{T_i} \leq 2.$$

We want to prove that

$$1 \leq \frac{T_{i+2}}{T_{i+1}} \leq 2.$$

By the recursive definition,

$$T_{i+2} = T_{i+1} + T_i.$$

Therefore

$$\frac{T_{i+2}}{T_{i+1}} = \frac{T_{i+1} + T_i}{T_{i+1}} = 1 + \frac{T_i}{T_{i+1}}.$$

Since $T_i > 0$ and $T_{i+1} > 0$, we have

$$\frac{T_i}{T_{i+1}} > 0.$$

Hence

$$1 + \frac{T_i}{T_{i+1}} \geq 1.$$

Also, from the induction hypothesis,

$$\frac{T_{i+1}}{T_i} \geq 1.$$

Since both terms are positive, this implies

$$\frac{T_i}{T_{i+1}} \leq 1.$$

So

$$1 + \frac{T_i}{T_{i+1}} \leq 2.$$

Combining the two inequalities gives

$$1 \leq \frac{T_{i+2}}{T_{i+1}} \leq 2.$$

This proves the induction step. Therefore, for all $i \geq 2$,

$$1 \leq \frac{T_{i+1}}{T_i} \leq 2.$$

3c

The sequence is defined by

$$a_1 = 5, \quad a_2 = 13, \quad a_n = 5a_{n-1} - 6a_{n-2} \quad \text{for } n > 2.$$

We prove by strong induction that

$$a_n = 2^n + 3^n$$

for all positive integers n .

First, for $n = 1$,

$$a_1 = 5$$

and

$$2^1 + 3^1 = 2 + 3 = 5.$$

So the formula is true for $n = 1$.

For $n = 2$,

$$a_2 = 13$$

and

$$2^2 + 3^2 = 4 + 9 = 13.$$

So the formula is also true for $n = 2$.

Now suppose that the formula is true for all positive integers smaller than some n , where $n > 2$. In particular, we assume that

$$a_{n-1} = 2^{n-1} + 3^{n-1}$$

and

$$a_{n-2} = 2^{n-2} + 3^{n-2}.$$

Using the recursive definition,

$$a_n = 5a_{n-1} - 6a_{n-2}.$$

By the induction hypothesis,

$$a_n = 5(2^{n-1} + 3^{n-1}) - 6(2^{n-2} + 3^{n-2}).$$

Expanding gives

$$a_n = 5 \cdot 2^{n-1} + 5 \cdot 3^{n-1} - 6 \cdot 2^{n-2} - 6 \cdot 3^{n-2}.$$

Now collect the powers of 2 and the powers of 3 separately:

$$a_n = (5 \cdot 2^{n-1} - 6 \cdot 2^{n-2}) + (5 \cdot 3^{n-1} - 6 \cdot 3^{n-2}).$$

For the first part,

$$5 \cdot 2^{n-1} - 6 \cdot 2^{n-2} = 10 \cdot 2^{n-2} - 6 \cdot 2^{n-2} = 4 \cdot 2^{n-2} = 2^n.$$

For the second part,

$$5 \cdot 3^{n-1} - 6 \cdot 3^{n-2} = 15 \cdot 3^{n-2} - 6 \cdot 3^{n-2} = 9 \cdot 3^{n-2} = 3^n.$$

Therefore

$$a_n = 2^n + 3^n.$$

This proves the induction step. Hence, for all positive integers n ,

$$a_n = 2^n + 3^n.$$